

SubnetIQ

Phase 0: Research, product specification and architecture

Research date: 4 October 2026. Status: specification complete; application implementation has not started. Prepared from the attached project brief and the public sources listed at the end of this report.

Executive decision

Build a complete IP addressing platform around a connected workflow: calculate a result, understand its reasoning, practice the underlying concept, turn it into a network plan, then save and share that plan. Preserve all features in the supplied brief across Phases 1-11. Additions below strengthen practical planning, correctness and continuity rather than expanding into unrelated enterprise administration.

Keep SubnetIQ as the working project name. A current networking services business already uses SubnetIQ Networks, so the name needs review before public branding or domain purchase. This research does not establish trademark rights or domain availability. Keep product naming and canonical URLs configurable so a later rename does not affect the network engine. [S20]

The proposed advantage is the quality of the complete workflow. This review does not establish that SubnetIQ will outperform every competitor, rank first in search, or attract paying customers. Those outcomes require a working product and comparative testing. We can make the intended advantage concrete through exact mathematics, coherent explanations, useful saved plans, accessible interaction, and reproducible exports.

1. Competitor research

This is a review of public product pages, official documentation, repositories and first-person issue reports. It is not a signed-in evaluation, a paid-plan audit, an accessibility audit, or a performance benchmark. "Not documented" means a capability was not established in the reviewed sources; it does not prove absence. The products below are relevant comparators, not a measured ranking by market share.

Competitor feature matrix

Product	Documented features and strengths	Boundary and implication for SubnetIQ
subnet-calculator.com	Detailed IPv4 outputs, binary bitmap, wildcard helper and separate CIDR calculator. The main page explicitly uses a classful model. [S01] [S02]	IPv6, persistent projects and adaptive practice were not documented. Default to classless input while retaining classes as historical teaching metadata.
SubnettingPractice	IPv4/IPv6 practice and calculators, explanatory material, and VLSM allocation from subnet names and host requirements. [S03] [S04]	Its IPv6 page limits display to the first 999 subnets. Saved project revisions and durable mastery analysis were not established. Connect learning progress to reusable plans and disclose preview limits.
CIDR.xyz	Interactive IPv4 bit display, CIDR base, usable endpoints, copy/share links and explanatory FAQ. [S05]	Current product explicitly describes IPv4. Requirements-based allocation and saved projects were not documented. Match its immediate feedback and visual clarity.
Visual Subnet Calculator, David C original	IPv4 split/join interaction, range and host information, and bookmark-based save/load. [S06]	The author describes IPv6 as future work. Add allocation labels, locked blocks and IPv6 planning, while preserving simple reversible interaction.
VisualSubnetCalc, modern fork	Notes, colors, import/export, share links, and standard/AWS/Azure/OCI modes. Its stated model keeps data under user control without server storage. [S07]	Cloud modes and colored subnet maps already exist. Our differentiation must extend to explanations, durable revision history and learner progress.
ipcalc, Jodies/kjokjo family	Scriptable local calculation and binary teaching output; the continuation repository explicitly supports IPv6. [S08]	Implementations and interfaces differ. Do not describe all ipcalc tools as IPv4-only. Retain exact, reusable results and test bulk calculation performance.
GestioIP online planner and full IPAM	Hierarchical IPv6 planning and CSV export; the full IPAM adds IPv4/IPv6 management, discovery, free ranges and auditability. [S09] [S10]	Online planning documentation shows only 20 final-level /64 entries before export. The deployed IPAM has a broader operational scope. Offer immediate guest access and clear navigation through large plans.
MXToolbox	Integrated DNS, mail and other diagnostics, result-to-next-test links, history, and a subnet calculator. [S11] [S12]	Hierarchical address planning and subnetting practice were not established in the reviewed calculator pages. Adopt useful handoffs between tools.
Network Toolbox native app	Publisher describes 50+ tools, including network diagnostics and security utilities with per-tool help. [S13]	The native app has platform and subscription considerations; it is not evidence that every feature exists on a website. Deliver consistent browser access and reference help.

Product	Documented features and strengths	Boundary and implication for SubnetIQ
subnetcalculator.appspot	Self-hosted IPv4/IPv6 suite with VLSM, overlaps, allocation tools, saved sessions, exports and REST API. [S14]	This is a strong planning comparator. A wide calculator catalog plus persistence alone is not a unique proposition. Learning continuity and polish must add value.
Cisco subnetting reference	Worked subnetting and VLSM examples provide a useful explanation benchmark. [S15]	This reference is educational documentation, not a directly comparable hosted planning application. Use standards-grounded original teaching content.

What users actually reported

Evidence	Interpretation	Product response
CIDR.xyz issue 65, January 2023, closed: a user found the distinction between all addresses and usable addresses confusing. [S16]	Historical evidence of semantic confusion, not a verified current bug.	Label complete range, conventional host range and policy-specific capacity separately.
David C issue 6, April 2021, open: a user requested locked allocations, visual marking and labels. [S17]	A concrete planning feature request from a user who liked the tool.	Store allocation purpose and state; preserve locked blocks when replanning.
David C issue 4, November 2020, open: a user requested IPv6. [S18]	Direct IPv6 demand in that product's tracker.	Make IPv6 part of calculation, planning, learning and exports.
kjokjo/ipcalc issue 27, March 2026, open: a user reported slower individual command execution in version 0.51 than 0.41 and warned that repeated script invocation would amplify the impact. [S19]	An individual report, not an independently reproduced benchmark or a statement about every ipcalc implementation.	Add reproducible performance fixtures and avoid recomputing whole plans on each keystroke.

This focused review did not establish a recurring current complaint pattern for every comparator. Claims such as “users hate the design,” “the tools are inaccurate,” or “there is no mobile support” are not justified by this evidence.

2. Features that strengthen the competitive proposition

These are product decisions and hypotheses to validate, not claims of exclusive features or proven willingness to pay.

Addition	Why it earns a place	Acceptance evidence
One result model across tools, lessons and projects	Users can carry a result forward without retyping or losing its assumptions.	Open a calculator result in a plan, explanation, export or AI attachment with identical normalized values.
Explicit address-allocation policies	A CIDR's mathematical size differs from LAN, point-to-point and provider-specific capacity.	Results and exports name the selected policy and show reservations separately.
Locked blocks, reservations and growth headroom	Real plans contain existing allocations and future requirements.	Replanning retains locks, prevents collisions and reports remaining space.
IPv4/IPv6 pairing and hierarchy	A site or VLAN can have both address families.	Related allocations share labels and purpose while maintaining independent arithmetic.
Plan revisions and visual comparisons	Users need to understand what changed and retain intent.	Show added, removed, moved and resized allocations with a restorable revision.
Exact aggregation and explicit covering summaries	A single summary can include addresses outside the intended input.	Exact output preserves the union; covering output lists additional ranges.
Capacity profiles for AWS, Azure and Google Cloud	Provider reservations affect practical host capacity. [S39] [S40] [S41]	Versioned, sourced rules distinguish relevant network types and exceptions.
Targeted practice after a mistake	Educational value comes from explaining and repairing a misconception.	Topic-tagged errors lead to an explanation and a relevant follow-up exercise.
Search, index navigation and bounded rendering for large plans	IPv6 spaces cannot be treated as lists of every address.	Show totals, displayed subset, jump controls and explicit export limits.
Reviewed data snapshots and rule provenance	Classifications and operational advice need a traceable basis.	Results identify the rule/dataset version and link to relevant source material.

3. Final feature specification

All families below are included in the complete delivery. The phase plan controls order; it does not move required features out of scope.

A. IPv4 suite

- Subnet calculator: network, broadcast where applicable, first/last conventional usable address, total and usable counts, mask, wildcard, CIDR, binary and hexadecimal representations, historical class and special-use classification.
- Equal subnet splitting by target prefix or a valid power-of-two count. A separate “allocate N equal blocks” operation may leave unused space when N is not a power of two.
- VLSM allocation from named segments and required host counts, with capacity validation, visual map, utilization and CSV/JSON/PDF exports.
- Exact route aggregation, single covering-supernet calculation and explicit reporting of extra covered space.
- Inclusive IP range to minimal CIDR list, and CIDR to inclusive range.
- Overlap, containment, duplicate and conflict analysis across multiple networks, including import preview.
- Wildcard/CIDR conversion and Cisco ACL generation, with explicit handling of noncontiguous wildcard masks rather than pretending every wildcard is one CIDR.
- Binary, decimal, hexadecimal and octal conversion with step-by-step explanations.
- Strict address validation and classification: RFC 1918, loopback, link-local/APIPA, multicast, CGNAT/shared space, documentation, benchmarking and other special-purpose ranges.
- Host reverse names and subnet reverse-delegation guidance for in-addr.arpa.
- Searchable/filterable /0 through /32 mask and host-capacity table, plus previous/next subnet navigation.

B. IPv6 suite

- Exact /0 through /128 parsing and calculation, address expansion/compression, prefix breakdown, inclusive range and address count.
- Global-unicast, ULA, link-local, multicast, loopback, unspecified, mapped and special-purpose classification.
- MAC-to-modified-EUI-64 calculation, including the inserted ff:fe and changed universal/local bit, with privacy context.
- Hierarchical allocation centered on /48, /56, /60 and /64 planning; labels, reservations, paired IPv4 networks, and valid point-to-point/host-route cases.
- IPv4-mapped conversion, NAT64 encoding/decoding and explanations, historical 6to4 material and dual-stack transition lessons.
- Full ip6.arpa host names and delegation guidance for nibble and non-nibble prefix boundaries.

C. Explanation and result system

Every mathematical tool will emit a deterministic structured explanation. IPv4 subnetting includes binary AND, network/host bits, borrowed bits relative to an explicit parent prefix, powers of two, magic-number and block-size reasoning. VLSM explains allocation order and alignment. IPv6 explains prefix bits and hexadecimal boundaries. Bandwidth and MTU/MSS tools explain units and assumptions. DNS lookups instead explain records and provenance; a hash report identifies its algorithm and encoding.

Results share copy, share, print, CSV, JSON and PDF controls. Scalars export as key/value rows; plans export as tables; PDFs include normalized inputs, assumptions and optional explanation steps. Exports derive from the result data model, not screenshots of the DOM. Very large theoretical spaces export compact range descriptors or an explicitly bounded selection; the UI never silently truncates a supposedly complete export.

Calculator links encode validated nonsecret inputs without requiring a saved account. Passwords and hash source text never enter URLs, exports, telemetry, saved history or AI context. Password reports may export the assessment without the original password; hash tools may export the resulting digest. Conversation export and native text sharing use an explicit included-message preview; the native sharing control has a copy-text fallback.

D. Network projects

Named projects contain networks, VLSM plans, labels, notes, purpose, VLAN identifiers, gateway choices, reservations and paired IPv4/IPv6 assignments. Add CSV/JSON import with validation and preview, visual space maps, locked allocations, growth settings, undo/redo, revisions, comparisons, duplicate/archive actions and read-only sharing.

Overlap is checked within a selected address-space context. Different sites or routing domains can intentionally reuse private ranges; the model must not reject all overlap globally. A context field explains the scope of each conflict check.

Default public sharing exposes a selected immutable snapshot. Links are revocable and can expire. Later private edits do not silently alter the snapshot. Guest planning remains available in memory; durable project saving and history require authentication.

E. Learning hub

Content targets are 220 original glossary entries and 12 complete course modules covering IP basics, binary mathematics, historical classes, CIDR, subnetting, VLSM, summarization, IPv6, NAT, DHCP, DNS and routing fundamentals. Each glossary entry has a category, definition, example and related terms. Switching, security and protocol terms are represented as requested.

Practice includes beginner, intermediate, advanced and exam modes, seeded random generation, timer, scores, streaks, explanations and topic-level error analysis. Include at least 160 original curated quiz questions alongside the generated questions. Persist attempts, history and weak-topic statistics for signed-in users. Guest sessions show current-session progress without storing an account history. Describe exam questions as original CCNA-style practice, not official exam questions or a passing guarantee.

Printable sheets cover CIDR, powers of two, private/special ranges, common ports and OSI/TCP-IP. Interactive learning includes a binary-octet toggler, subnet-block visualizer and address-space bar map, each with a keyboard-operable and textual equivalent.

F. Networking and cybersecurity toolkit

- Ports and protocols: searchable, categorized static references with TCP/UDP distinctions, aliases, source metadata and security notes. Reviewed service-specific advice is identified separately from general guidance; a port number is not treated as proof of a vulnerability.
- Interactive OSI and TCP/IP models, with encapsulation and protocol examples.
- MAC formatting and OUI lookup using an explicitly labeled subset; local/randomized address detection and clear “not in this dataset” results.
- Backend DNS lookup for A, AAAA, MX, TXT, NS, CNAME and PTR, with record interpretation, provider, retrieval time and cache status.
- Registration/WHOIS and IP information: structured RDAP as the primary route, plus a tightly allowlisted legacy WHOIS fallback where supported. Identify the actual information source and unavailable/redacted fields.
- Firewall output for iptables, UFW, Cisco ACL and a pfSense-style rule worksheet. Select family, direction, protocol, ports and source/destination explicitly. Do not label a worksheet as a tested native pfSense import or apply rules remotely.
- Home, SMB, campus and data-center design templates with complete sample subnet plans, documented assumptions and editable segment purposes.
- Threat/defense glossary covering ARP spoofing, MITM, DDoS, scanning, VLAN hopping and DNS poisoning.
- Local-only password-strength assessment and SHA-256/SHA-384/SHA-512 digest generation; lazy-load larger strength dictionaries and label estimates accurately.
- HTTP header and TLS learning references; CVSS 4.0 and common-vulnerability cards with versioned examples. CVSS severity is distinguished from an organization's complete risk assessment. [\[S34\]](#)
- Searchable command library for ping, tracert/traceroute, nmap, netstat, ip, ifconfig, dig, nslookup and tcpdump, with platform and purpose labels. Commands are examples for the user's environment.
- Bandwidth and transfer-time calculation with bits/bytes, decimal/binary units and efficiency assumptions; MTU/MSS calculation with family, headers, options and encapsulation overhead.

The additional toolkit emphasis is explanation and planning utility. Live operations remain the requested lookup services; the command library and firewall generators do not execute operations against a user's network.

G. AI assistant

Deliver a floating widget and full-page assistant with suggested prompts, streaming markdown, code blocks, copy, stop/retry, connection/error states, conversation controls and export/share. Users can explicitly attach the current validated calculation. The system prompt specializes in networking, subnetting and defensive security education; numerical explanations refer to the deterministic engine rather than replacing it.

The backend provides `/api/ai/chat` and a provider interface selected by environment configuration. Phase 4 supplies a functioning deterministic streaming mock and explicit OpenAI, Anthropic and Gemini adapter stubs. The mock is prominently labeled “AI not configured - demo response.” Phase 9 completes the assistant experience and one live adapter, with OpenAI as the default integration choice. Adding the selected provider key and model then enables live responses; without credentials, the labeled mock works. Anthropic and Gemini remain explicit adapter stubs

until separately implemented. A key is never represented as activating an unfinished adapter. Live-provider smoke testing is identified separately from deterministic CI tests and requires credentials.

Selected messages can be shared as text through the native sharing control or copy fallback after preview. Guest mock conversations are ephemeral. Authenticated conversations and messages persist in Supabase. Account quotas and usage records, separate guest request limits, maximum context/output sizes, cancellation and partial-stream states are part of the API contract. External-provider usage is never hidden behind an unlabeled mock fallback.

H. Accounts and persistence

Supabase Auth supports email/password, email verification/recovery, magic links, Google OAuth and GitHub OAuth, with configured provider credentials and redirects. Profiles contain preferences and display information. Saved calculations, projects, networks, plans, quiz history, favorites, feedback and AI conversations have explicit ownership.

Guest tools remain useful without login. Login enables durable saving and history. Add account data export/deletion and session controls. Supabase Storage has a concrete role: private user-owned export artifacts and optional profile assets, with ownership policies and time-limited downloads. Ordinary guest downloads are generated locally.

I. Experience, discovery and marketability

Use a restrained SaaS design with clear typography, monospace IP values, Lucide icons, responsive tool panels, light/dark/system themes, subtle Framer Motion transitions and reduced-motion support. Include Ctrl/Cmd+K global search, breadcrumbs, contextual related tools, loading skeletons, error boundaries, empty states, toasts and visible keyboard focus. Target WCAG 2.1 AA with actual keyboard and assistive-technology review.

English is complete. Provide locale structures for Amharic, Arabic, French and Spanish, Arabic RTL layout handling and suitable font fallbacks. Untranslated languages use an honest fallback; they are not presented as completed translations. Explanations and content references use translation keys and stable IDs.

The landing page contains the hero, a live mini calculator, feature grid, categories, FAQ, footer, and the requested testimonial/pricing sections. Their placeholder status remains explicit until actual quotes and commercial terms exist. No invented customers, endorsements or prices.

Build individual tool landing pages, a tool dashboard, learning routes, projects, assistant, About, Contact, Privacy and Terms. Include five complete original markdown articles: CIDR and host counts; a worked VLSM plan; practical IPv6 /64 planning and exceptions; wildcard masks and ACLs; and reading DNS records during troubleshooting. Legal-page operator details and actual retention/vendor practices must match the deployed service.

SEO includes unique metadata, canonical URLs, Open Graph/Twitter cards, appropriate JSON-LD, sitemap and robots controls. Pre-render public pages to HTML, then hydrate calculators; keep private projects and share-token pages out of indexing. Google explicitly recommends considering pre-rendering/server rendering and meaningful status codes for JavaScript sites. [S38]

PWA support caches the app shell, calculators and selected reference lessons for offline use. It excludes authenticated responses, private projects, share-resolution responses, AI conversations and live lookup results. Include a visible offline state and an update flow. Analytics is an optional hook with consent behavior suited to the selected provider; calculation inputs and secrets are excluded.

4. Network correctness contract

The shared engine uses exact integers and range arithmetic. BigInt is the internal representation; JSON carries large counts as decimal strings. No calculation enumerates an entire IPv4 or IPv6 address space. Parsing is strict and consistent across browser and API. Noncanonical network inputs are either explicitly normalized with feedback or rejected in strict import mode.

Case	Required behavior
IPv4 /0	Total 4,294,967,296; full IPv4 range; conventional host arithmetic is not a claim that all addresses are assignable. [S45]
IPv4 /31	Two endpoints in RFC 3021 point-to-point mode; no directed-broadcast address. Name the assumption. [S21]
IPv4 /32	One address/host route; first and last equal; broadcast not applicable. [S45]
IPv6 /0 through /128	Accept every valid mathematical prefix; no broadcast and no IPv4-style subtraction of two. [S22]
IPv6 /48 to /64	Exactly 65,536 child /64s; display or allocate them without expanding every host.
IPv6 /127 and /128	Explain inter-router point-to-point /127 and single-address /128 cases separately from /64 LAN defaults. [S23] [S44]
IPv6 formatting	RFC 5952 output: lowercase, removed leading zeros, longest zero run, first run on ties, no compression of a single zero field. [S26]
Classification	Longest-prefix IANA matching, mixed-range detection and distinct purpose/reachability flags. Actual reachability is not inferred from a registry label. [S24] [S25]
EUI-64	Explain the conversion while acknowledging modern privacy recommendations against default stable MAC embedding. [S27]
Reverse DNS	Distinguish a host's reverse name from authoritative delegation; include RFC 2317 guidance for classless IPv4 and nibble-aware IPv6 handling. [S28] [S43]
NAT64 and mapping	Separate mapped-address representation from translation layouts; implement every supported RFC 6052 prefix format correctly. [S29]
6to4	Teach historical context and the specific deprecation of the anycast relay mechanism, without overstating the RFC. [S30]

For VLSM, “optimized” means deterministic largest-first aligned allocation using the smallest feasible block under the chosen capacity policy, with explicit unallocated space. With locks and exclusions, report infeasibility and tradeoffs; do not claim a proof of global network-design optimality.

Worked acceptance fixture: a 192.168.10.0/24 parent, conventional LAN policy for the first three segments, RFC 3021 for the link, and zero extra reservations/growth.

Segment	Requested endpoints	Allocation	Capacity under selected policy
Engineering	80	192.168.10.0/25	126
Operations	40	192.168.10.128/26	62
Guest	20	192.168.10.192/27	30
Router link	2	192.168.10.224/31	2

The remaining range is 192.168.10.226-192.168.10.255: 30 addresses, representable as .226/31, .228/30, .232/29 and .240/28. No existing segment overlaps another. Host requirements must explicitly state whether gateway/infrastructure endpoints are included.

5. Technical architecture

Stack decisions

Use the requested .eslintrc through an active flat-config compatibility entry rather than shipping an ignored configuration. ESLint documents FlatCompat for translating that format. Versions and peer dependencies are verified before the Phase 1 lockfile is committed. [\[S37\]](#)

Layer	Selected approach	Purpose
Workspace/runtime	pnpm workspaces, TypeScript strict mode, Node.js 24 LTS	Reproducible builds and supported server runtime. Node currently lists 24 as LTS. [S35]
Web	React 18, Vite, React Router, Tailwind CSS, Radix UI, Framer Motion, Lucide	Preserve the mandatory stack; pin a mutually compatible dependency set in Phase 1. Vite supports the selected runtime. [S36]
State/forms	Zustand for transient UI; TanStack Query for server state; Zod contracts	Clear separation between local interaction, cached remote data and validation.
API	Node/Express TypeScript REST with streaming responses	Account operations, lookups, shares, quotas and provider integration.
Pure engine	packages/netcalc	Exact networking algorithms and structured explanation traces shared by web and API.
Contracts	packages/shared	Serializable schemas, DTOs, error codes, content IDs and result definitions.
Persistence	Supabase PostgreSQL, Auth and private Storage	Identity, relational ownership, history and stored artifacts.

Layer	Selected approach	Purpose
Content	Versioned JSON and trusted markdown	Searchable references, original learning material, deterministic quiz content and pre-rendered pages.
Testing	Vitest/RTL, Jest/Supertest, Playwright, SQL policy tests	Verify mathematics, user behavior, API boundaries and real database access.
Deployment	Vercel web, Render API, hosted Supabase; Docker local/self-host path	Match the requested deployment files. Railway is an optional documented API alternative.

Component responsibilities and trust boundaries

The browser runs ordinary calculations, visualizations, practice generation and local exports. It talks directly to Supabase Auth for sign-in. For protected API requests it sends the Supabase access token explicitly. Express validates token signature, issuer, audience and expiry, then uses user-scoped database access so RLS remains effective.

The API reuses netcalc when a server-side calculation or validation is needed. Ordinary arithmetic does not depend on API availability. netcalc imports no UI, database, provider client or browser API. Runtime timestamps and persistence metadata are added outside the pure engine. Types and validation contracts have one source of truth; generated database types remain separate from public DTOs.

Long calculations and bounded bulk exports run in a Web Worker to keep inputs responsive. A tool registry defines each route, name, search aliases, input schema, related lessons, example and result/export capabilities. This supports global search, related-tool navigation and consistent pages without duplicating routing logic.

API surface

Route family	Responsibility
/api/health and /api/ready	Process and dependency readiness without exposing secrets.
/api/calculate/:tool	Optional parity endpoint using the shared engine and limits.
/api/projects, networks, plans and revisions	User-owned CRUD, optimistic concurrency and plan snapshots.
/api/saved-calculations and /api/favorites	Saved results and shortcuts.
/api/lookups/dns and /api/lookups/ip-info	Validated, cached public-information lookups.
/api/quiz-attempts and /api/practice/stats	Validated attempts, deterministic grading and signed-in progress.
/api/ai/chat and /api/ai/conversations	Streaming mock/provider contract, conversation data and usage.
/api/projects/:id/shares and /api/share/:token	Owner-managed grants and minimal read-only snapshot resolution.
/api/exports and /api/feedback	Optional private export storage and feedback handling.

Document request/response schemas and error codes in OpenAPI. Use pagination, bounded payloads, consistent errors, request IDs and explicit schema/engine versions. Streams distinguish started, delta, completed, canceled and failed states; cancellation ends upstream work and records partial usage accurately.

6. Data model and ownership

Table group	Tables	Key rule
Identity/preferences	profiles	Profile ID references Supabase auth.users; users can edit only permitted own fields.
Saved work	saved_calculations, projects, saved_networks, vlsm_plans, vlsm_segments	Owner access and child-to-parent ownership consistency.
Plan history/sharing	project_revisions, project_shares	Immutable snapshots; owner-managed, revocable token grants.
Learning	quiz_attempts, practice_stats	Attempts belong to the current user; stats derive from validated attempts.
Assistant	ai_conversations, ai_messages, ai_usage_events	Conversation ownership propagates to messages; provider usage is server-recorded.
Preferences/support	favorites, feedback	User-scoped retrieval; tightly controlled feedback submission and administrative access.
Internal service data	private.lookup_cache, private.api_usage_buckets	No browser table access; bounded TTL cache and atomic cross-instance quotas.

Use PostgreSQL cidr/inet types where they express networks and addresses correctly, JSONB for versioned tool results and snapshots, and exact numeric/string boundaries for large counts. Index ownership, project foreign keys, creation times and lookup expiry; constrain names, sizes, relationships and supported schema versions. A shared parent/owner constraint prevents a user-owned row from referencing another user's project.

Enable RLS and explicit grants on exposed user-data tables. UPDATE needs checks on both the old and new row. Never use a broad anonymous project-read policy for public links. Supabase documents that privileged service access can bypass RLS; it is excluded from ordinary user CRUD. [\[S33\]](#)

Share tokens are high-entropy, stored as hashes and resolved through a narrowly scoped operation to a limited snapshot projection. Revocation stops future server resolutions; copies already downloaded remain with their recipients. Apply expiry/revocation checks on each server resolution, redact tokens from logs, and set noindex, Referrer-Policy: no-referrer and Cache-Control: no-store, and exclude shared responses from service-worker caches. If a privileged SQL function is required, fix its search_path and restrict execution; prefer invoker security where possible. [\[S42\]](#)

Migration delivery includes tables, policies, indexes, triggers, grants, Storage policies, seed data and ownership tests for anonymous, owner, unrelated user and share-recipient cases. Include tested account-deletion behavior so database rows and private objects do not become orphaned.

7. Backend and application security

Lookup connection destinations are controlled independently of the hostname or IP being investigated. DNS requests go to a configured public resolver. Returned A/AAAA/CNAME/MX values are data and are never followed as arbitrary URLs. A DNS record may legitimately contain a private address; displaying it is different from connecting to it.

RDAP authority selection uses reviewed, cached IANA bootstrap information and approved HTTPS registry origins. Legacy WHOIS support uses fixed allowlisted endpoints, bounded queries and no shell command interpolation. Accept no user-supplied upstream URL, port, resolver or executable. Disable automatic redirect/referral following; validate any approved follow-up again. Pin or validate actual connection destinations against DNS changes, including disallowed IPv4/IPv6 loopback, private, link-local and metadata targets. These choices implement the requested allowlist approach and OWASP guidance. [\[S31\]](#) [\[S32\]](#)

Use request, response-size, duration, concurrency and quota limits; TTL/negative caching; distinct invalid-input/no-record/timeout/rate-limit/upstream-failure states; and text-safe output rendering. Production rate/usage counters are atomic across API instances using PostgreSQL; development-only in-memory limits are clearly separated.

Apply Helmet, explicit CORS origins, content security policy, startup environment validation, centralized errors, structured redacted logs and trusted-proxy settings matched to deployment. API authentication uses explicit bearer tokens rather than ambient session cookies; cookie-based auth would require a separate CSRF design. OAuth redirect/state/PKCE behavior is validated as part of auth delivery. Do not treat CORS itself as authentication or CSRF protection.

Disable raw HTML in AI markdown, validate rendered link schemes and sanitize any deliberately supported markup. Imported data has size and row limits; CSV exports protect against spreadsheet formula injection. Password/hash inputs and raw private plans are excluded from logs and analytics. AI can use explicitly selected context, but cannot authorize privileged database or network operations.

No service-role keys or provider secrets enter frontend bundles or VITE-prefixed variables. CI checks lockfiles, dependency advisories, accidental secrets and required files. Deployments use TLS, non-root containers where applicable, explicit health checks and bounded resources.

8. User experience and commercial decisions

The primary navigation is Tools, Learn, Practice, Projects, Reference and Assistant. The tool shell presents inputs and immediate results first, then explanation and related actions. On wide screens the explanation can sit alongside results; on mobile it follows in a clear reading order. Visual maps use labels and shapes as well as color. Large result tables support keyboard interaction and meaningful mobile layouts.

Suggested visual direction: deep navy dark surfaces, light neutral surfaces, a restrained teal/blue accent, generous spacing and tabular/monospace numerals for address data. Actual tokens must pass contrast checks in both themes. Motion should explain a change in allocation or state and respect reduced-motion preferences.

Keep core calculations and basic learning available to guests. Account quotas for saved projects and later live AI are configurable. Paid team features and billing are not silently added to the current specification; the requested pricing section remains a clearly labeled commercial placeholder. Product research should later measure successful task completion, return use and willingness to pay with students, engineers and instructors rather than assuming demand.

9. Verification and release gates

Every exported mathematical function receives normal, boundary and invalid-input tests. Add independent known-answer fixtures and property tests for parse/format round trips, split/merge conservation, exact range covers, child containment, non-overlap, classification precedence and deterministic explanations. Explicit cases include /0, /31, /32, IPv6 /0 and /128, leading-zero ambiguity, embedded IPv4, noncontiguous masks, mixed address families, duplicates and exhausted plans.

Backend Jest/Supertest tests exercise validation, authorization, quotas, cache expiry, upstream failure, disabled redirects, destination checks and streaming cancellation. Supabase tests use real local policies with distinct identities; mocked frontend auth is not evidence that RLS works. Frontend Vitest/RTL tests focus on forms, keyboard interaction, explanation toggles and observable state transitions.

Playwright covers a guest calculation, saving/reopening a plan, denied cross-user access, share revocation, offline calculator operation, a practice session and mock AI streaming. Use deterministic providers in CI; separately identify credential-dependent live-provider and OAuth verification before launch.

Targets: Lighthouse 90+ on agreed production-build routes, usable mobile input and map navigation, WCAG 2.1 AA, no unresolved critical access-control or network-math failures, no silent export truncation, and complete required-file/content coverage. These are acceptance targets, not measurements already achieved. Record test environments and results with each phase.

10. Proposed monorepo folder tree

This is the intended structure, not a claim that these application files already exist. Feature directories contain complete modules as their phases are delivered.

```
subnetiq/  
  apps/  
    web/  
      src/  
        app/  
          router.tsx  
          providers.tsx  
          tool-registry.ts  
        components/  
          ui/  
          layout/  
          results/  
          visualizers/  
        features/  
          ipv4/  
          ipv6/  
          planners/  
          projects/  
          learning/  
          practice/  
          toolkit/  
          assistant/  
          auth/  
          marketing/  
        lib/  
          api/  
          supabase/
```

```
    exports/  
    i18n/  
    seo/  
  locales/  
    en/  
    am/  
    ar/  
    fr/  
    es/  
  stores/  
  styles/  
  workers/  
  test/  
public/  
  icons/  
  fonts/  
  robots.txt  
  sitemap.xml  
  manifest.webmanifest  
e2e/  
.env.example  
Dockerfile  
index.html  
package.json  
vite.config.ts  
vitest.config.ts  
playwright.config.ts  
tsconfig.json  
tsconfig.app.json  
tsconfig.node.json  
api/  
  src/  
    config/  
    middleware/  
    routes/  
    services/  
      lookups/  
      projects/  
      sharing/  
      practice/  
      ai/  
        providers/  
        prompts/  
      repositories/  
      security/  
      observability/  
    app.ts  
    server.ts  
  tests/  
    .env.example  
    Dockerfile  
    package.json  
    jest.config.ts  
    tsconfig.json  
packages/  
  netcalc/  
    src/  
      ipv4/  
      ipv6/  
      ranges/  
      vlsm/  
      allocation/
```

```
classification/
reverse-dns/
mac/
conversions/
bandwidth/
mtu/
explanations/
tests/
  fixtures/
  properties/
package.json
tsconfig.json
vitest.config.ts
shared/
src/
  schemas/
  types/
  errors/
  result-model/
package.json
tsconfig.json
supabase/
  migrations/
  tests/
  seed.sql
  config.toml
data/
  ports.json
  protocols.json
  glossary.json
  oui-subset.json
  rfc-index.json
  special-ipv4.json
  special-ipv6.json
  cloud-profiles.json
  quiz-bank.json
  network-templates.json
  sources.json
content/
  courses/
  blog/
  cheatsheets/
  legal/
scripts/
  validate-content.ts
  update-reference-data.ts
  generate-seo.ts
  verify-required-files.ts
  package-release.ts
docs/
  phase-0.md
  architecture.md
  environment.md
  api.openapi.yaml
  calculation-semantics.md
  data-provenance.md
  security-model.md
  deployment.md
  testing.md
  screenshots/
.github/
  workflows/
```

```
ci.yml
.husky/
  pre-commit
.gitignore
.gitattributes
.editorconfig
.nvmrc
.prettierrc
.prettierrignore
.eslintrc
eslint.config.mjs
lint-staged.config.mjs
package.json
pnpm-workspace.yaml
pnpm-lock.yaml
tsconfig.base.json
docker-compose.yml
vercel.json
render.yaml
README.md
LICENSE
THIRD_PARTY_NOTICES.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
SECURITY.md
CHANGELOG.md
```

Both `.env.example` files are complete and comment-free; `docs/environment.md` explains every variable, visibility, default, requirement and deployment location. Source/configuration files contain no comments as requested. The required `.gitignore` covers `node_modules`, `dist/build`, environment and local override files, logs, coverage, `.DS_Store`, local editor settings, `.turbo`, `.vercel`, `.supabase` and temporary files; it explicitly preserves the example environment files.

Docker Compose starts web and API with documented connections to hosted Supabase or the local Supabase CLI stack. It will not misleadingly substitute a bare PostgreSQL container for Supabase Auth and Storage. Root Vercel/Render configuration documents the monorepo build contexts. Source licensing is recorded explicitly; third-party content is redistributed only under applicable terms. The final archive includes the complete source and lockfile, excluding secrets, dependencies and disposable build caches.

11. Delivery phases and completion criteria

Phase	Complete delivery	Exit condition
0	Research, matrix, scope, architecture and folder tree	This reviewable report; no application code.
1	Monorepo, configs, strict TS, tooling, env documentation, root files and initial CI	Reproducible install, usable scripts, lint/typecheck and meaningful build checks.
2	All netcalc algorithms, explanation traces and tests	Normal/boundary/property fixtures pass; no unsafe integer truncation.
3	Supabase migrations, RLS, Storage, seed and grants	Fresh reset works; cross-user/parent ownership and sharing boundaries pass.
4	API, security, lookups, caches, quotas and streaming mock/provider contracts	Jest/Supertest and lookup/authorization failure cases pass.
5	Design system, routing, themes, i18n/RTL, search and auth	Responsive keyboard-operable shell and complete configured auth flows.
6	IPv4/IPv6 tools, visual planners, projects and all result controls	Calculations, saves, imports, exports and sharing behave consistently.
7	Courses, 220-entry glossary, practice, quiz bank and sheets	Content validates; grading, progression and signed-in history work.
8	Every named networking/cyber tool and four design templates	Local tools and backend lookup interfaces pass their relevant checks.
9	Floating/full-page assistant, contextual streaming, history and one live provider adapter	Mock, cancellation, persistence and quotas work; adapter tests pass and credential-dependent live verification is reported separately.
10	SEO, PWA, five articles, legal/contact/about, analytics and performance work	Public HTML, offline behavior, accessibility and performance reports are verified.
11	Docker, CI/CD, deployment guides, final README and ZIP	Clean install/build/test/package; complete file checklist and deployment commands.

Each implementation phase will finish with what changed, how it was verified, complete files, and any real configuration dependency. Files shown in chat will have their full path above their complete code block. No omitted-code placeholders or truncated files will be presented as finished work. Explicitly permitted mock/provider, pricing, testimonial and screenshot states remain visibly identified.

Launch-specific inputs are the final public name/domain, Supabase project configuration, OAuth credentials, email delivery settings, a live AI provider when integration is requested, actual

operator/contact information, and hosting accounts. They do not block Phase 1. No account creation, purchase or deployment is part of this Phase 0 work.

Phase 0 is complete. Type “continue” to begin Phase 1: monorepo setup, configuration, environment files and tooling.

Sources

The following public sources were retrieved or verified on 4 October 2026. Product descriptions are vendor or maintainer documentation; issue reports are individual user evidence. Architectural choices and acceptance targets are proposals made for this project.

- [S01] subnet-calculator.com, main calculator. <https://www.subnet-calculator.com/>
- [S02] subnet-calculator.com, CIDR calculator. <https://www.subnet-calculator.com/cidr.php>
- [S03] SubnettingPractice, VLSM calculator. <https://subnettingpractice.com/vlsm.html>
- [S04] SubnettingPractice, IPv6 calculator. <https://subnettingpractice.com/ipv6-subnet-calculator.html>
- [S05] CIDR.xyz, public calculator. <https://cidr.xyz/>
- [S06] David C, Visual Subnet Calculator. <https://www.davidc.net/networking/visual-subnet-calculator>
- [S07] VisualSubnetCalc, current product and design principles. <https://visualsubnetcalc.com/>
- [S08] kjokjo/ipcalc, official continuation repository. <https://github.com/kjokjo/ipcalc>
- [S09] GestioIP, hierarchical addressing plan guide. https://www.gestioip.net/docu/hierarchical_address_plan.html
- [S10] GestioIP, full IPAM features. <https://www.gestioip.net/>
- [S11] MXToolbox, SuperTool. <https://mxtoolbox.com/SuperTool.aspx>
- [S12] MXToolbox, subnet calculator. <https://mxtoolbox.com/SubnetCalculator.aspx>
- [S13] Network Toolbox, publisher's App Store listing. <https://apps.apple.com/us/app/network-toolbox-net-security/id651691453>
- [S14] SubnetCalculator.app, official documentation. <https://docs.subnetcalculator.app/>
- [S15] Cisco, IP addressing and subnetting guide. <https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13788-3.html>
- [S16] CIDR.xyz, issue 65, closed user report. <https://github.com/yuvadm/cidr.xyz/issues/65>
- [S17] David C subnets, issue 6, allocation-lock request. <https://github.com/davidc/subnets/issues/6>
- [S18] David C subnets, issue 4, IPv6 request. <https://github.com/davidc/subnets/issues/4>
- [S19] kjokjo/ipcalc, issue 27, reported performance regression. <https://github.com/kjokjo/ipcalc/issues/27>
- [S20] SubnetIQ Networks, existing commercial name. <https://www.subnetiq.net/>
- [S21] IETF RFC 3021, IPv4 /31 point-to-point links. <https://www.rfc-editor.org/rfc/rfc3021.html>
- [S22] IETF RFC 4291, IPv6 addressing architecture. <https://www.rfc-editor.org/rfc/rfc4291.html>
- [S23] IETF RFC 6164, /127 inter-router links. <https://www.rfc-editor.org/rfc/rfc6164.html>
- [S24] IANA, IPv4 special-purpose registry. <https://www.iana.org/assignments/iana-ipv4-special-registry/>
- [S25] IANA, IPv6 special-purpose registry. <https://www.iana.org/assignments/iana-ipv6-special-registry/>
- [S26] IETF RFC 5952, IPv6 text representation. <https://www.rfc-editor.org/rfc/rfc5952.html>

- [S27]** IETF RFC 8064, stable IPv6 interface identifiers. <https://www.rfc-editor.org/rfc/rfc8064.html>
- [S28]** IETF RFC 2317, classless reverse delegation. <https://www.rfc-editor.org/rfc/rfc2317.html>
- [S29]** IETF RFC 6052, IPv4-embedded IPv6 addressing. <https://www.rfc-editor.org/rfc/rfc6052.html>
- [S30]** IETF RFC 7526, anycast 6to4 relay deprecation. <https://www.rfc-editor.org/rfc/rfc7526.html>
- [S31]** OWASP, SSRF prevention guidance.
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- [S32]** IETF RFC 9224, authoritative RDAP service discovery. <https://www.rfc-editor.org/rfc/rfc9224.html>
- [S33]** Supabase, Row Level Security. <https://supabase.com/docs/guides/database/postgres/row-level-security>
- [S34]** FIRST, CVSS 4.0 specification. <https://www.first.org/cvss/v4.0/specification-document>
- [S35]** Node.js, current release/support listing. <https://nodejs.org/en/about/previous-releases>
- [S36]** Vite, getting started and runtime requirements. <https://vite.dev/guide/>
- [S37]** ESLint, configuration migration and FlatCompat. <https://eslint.org/docs/latest/use/configure/migration-guide>
- [S38]** Google Search Central, JavaScript SEO basics.
<https://developers.google.com/search/docs/crawling-indexing/javascript/javascript-seo-basics>
- [S39]** AWS, VPC subnet sizes and reserved addresses.
<https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html>
- [S40]** Microsoft, Azure Virtual Network FAQ.
<https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-faq>
- [S41]** Google Cloud, VPC subnets and reservation rules. <https://docs.cloud.google.com/vpc/docs/subnets>
- [S42]** Supabase, database-function security. <https://supabase.com/docs/guides/database/functions>
- [S43]** IETF RFC 3596, IPv6 DNS extensions. <https://www.rfc-editor.org/rfc/rfc3596.html>
- [S44]** IETF RFC 7421, IPv6 /64 boundary analysis. <https://www.rfc-editor.org/rfc/rfc7421.html>
- [S45]** IETF RFC 4632, CIDR and address-block sizes. <https://www.rfc-editor.org/rfc/rfc4632.html>